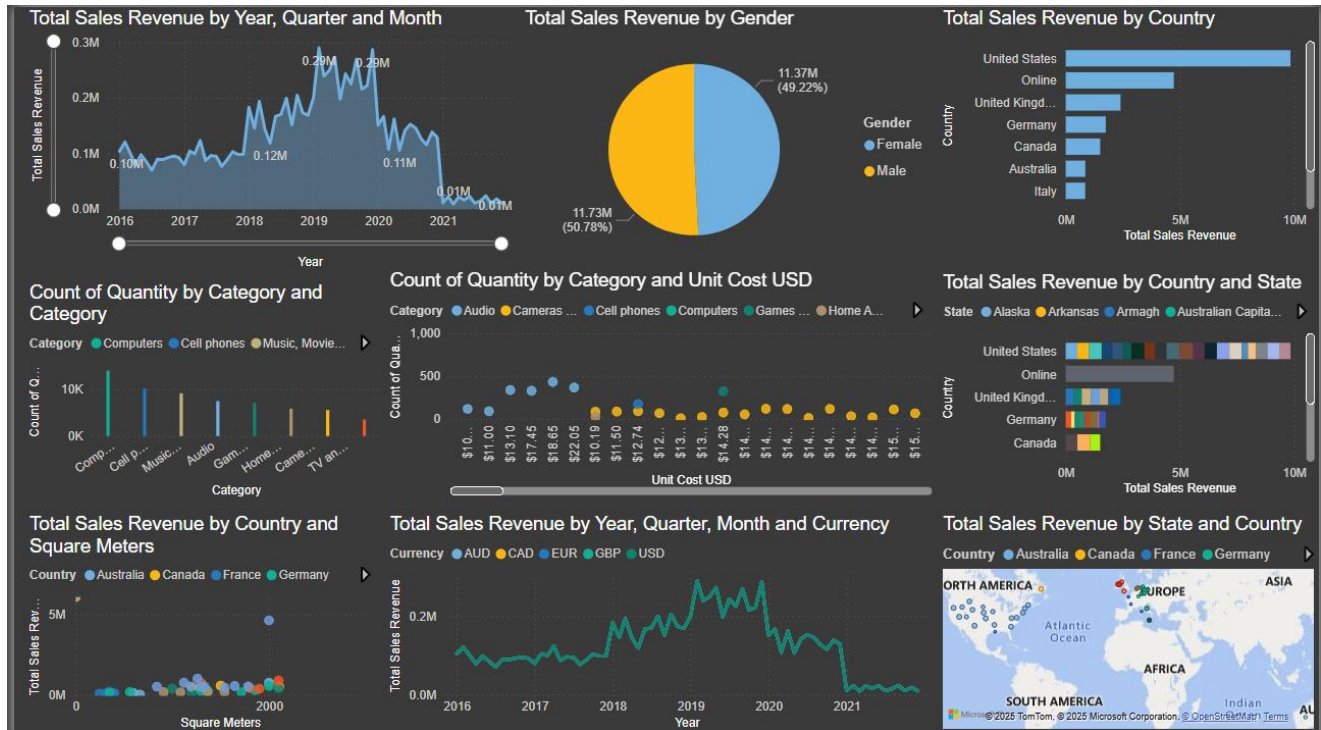


1. Global Electronics retailer: (Multiple Table Dashboard using joins and DAX script in MS Power BI)

Dataset Source:

https://mavenanalytics.io/dataplayground?dataStructure=Multiple%20tables&order=date_added%2Cdesc&page=2&pageSize=5

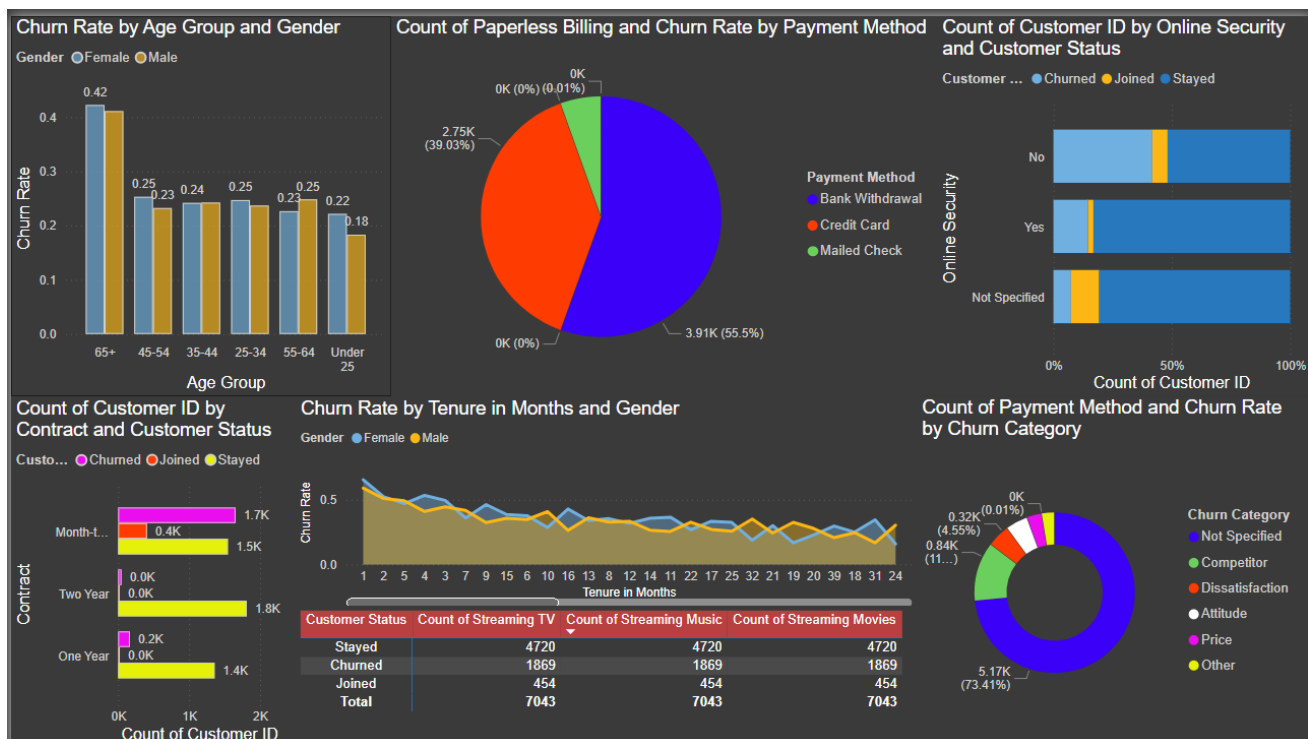


2. Telecom Customer Churn Data:

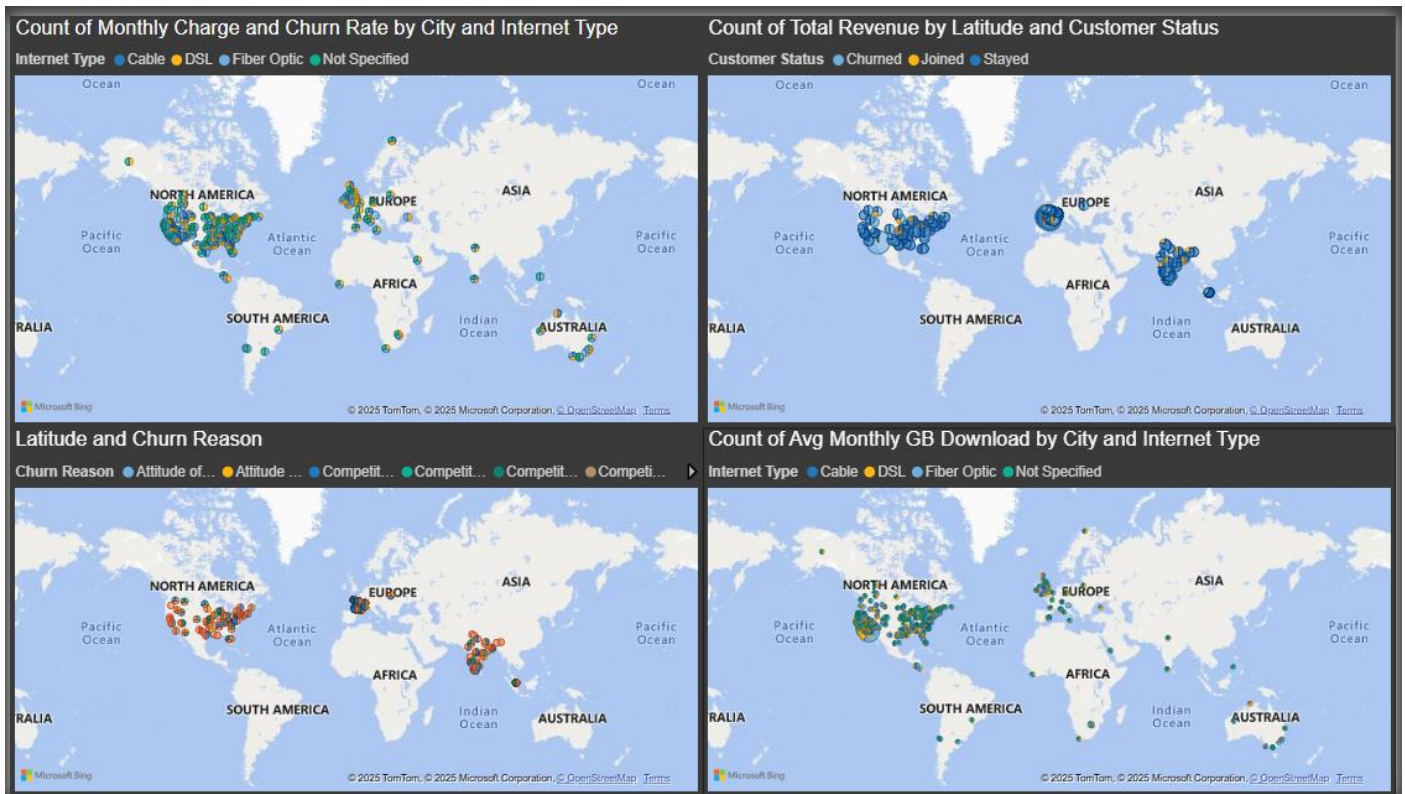
Dataset Source: https://mavenanalytics.io/data-playground?order=date_added%2Cdesc&search=telecom

Tools Used: MS Power BI.

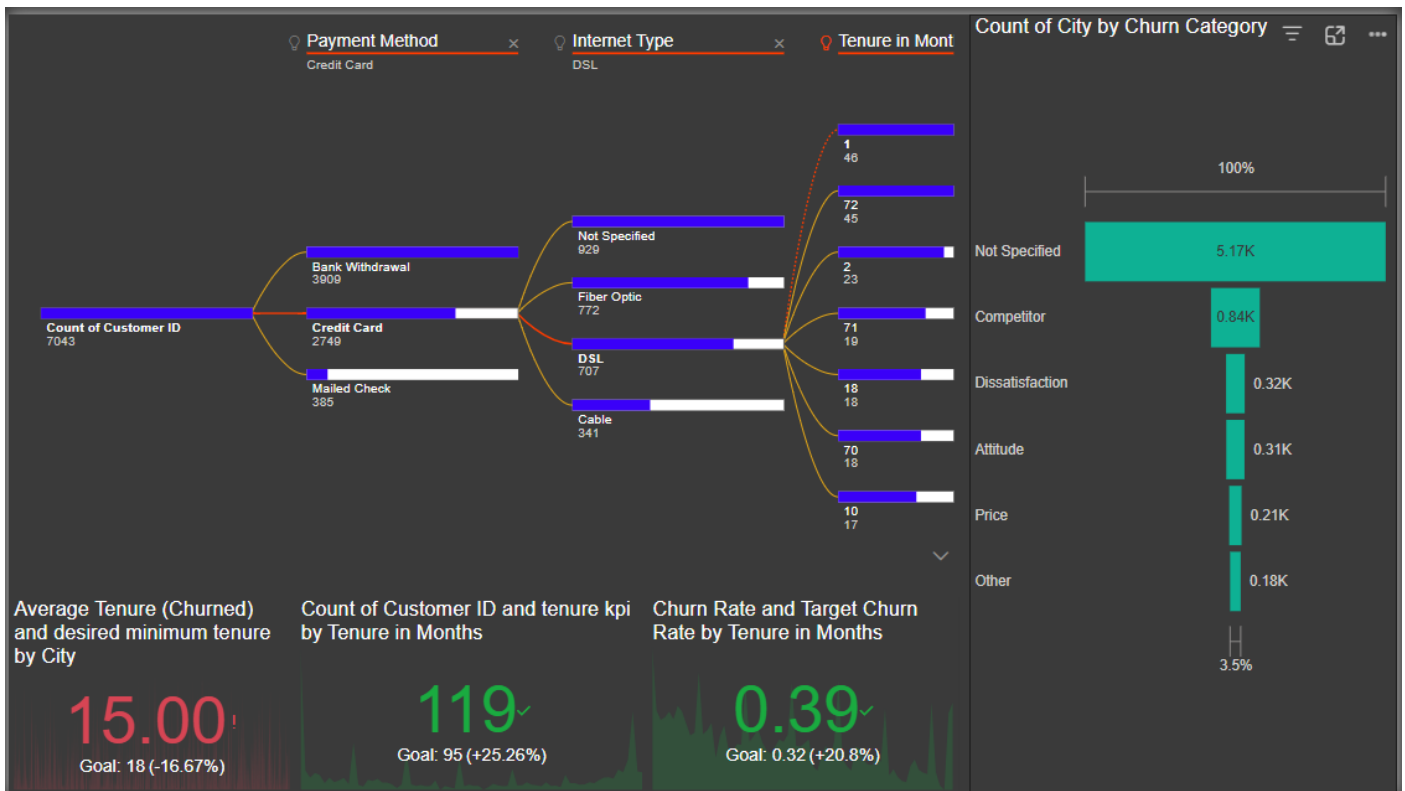
a. General Insights



b. Geospatial Data

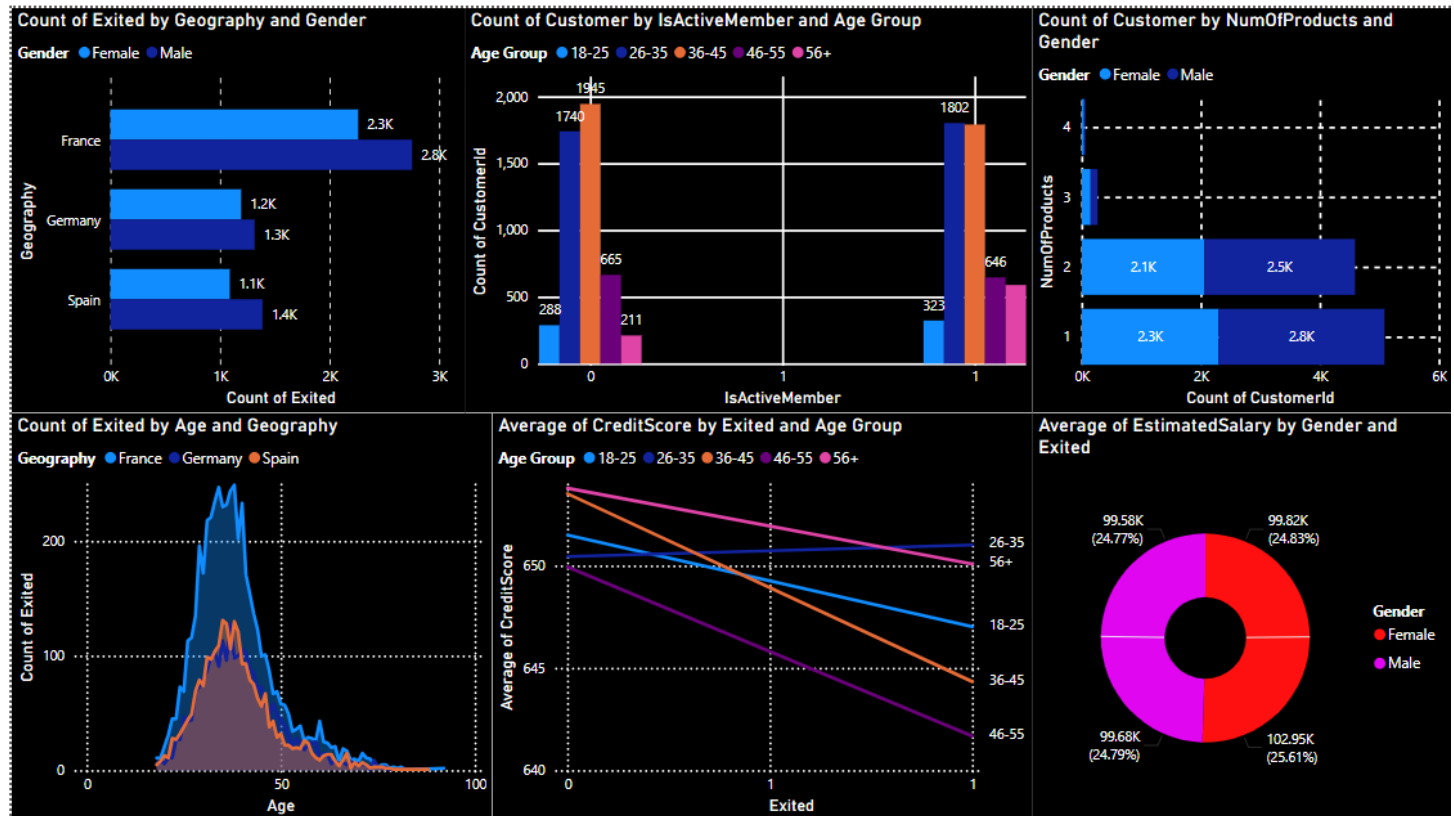


c. KPIs



3. Europe Bank Customer Churn data:

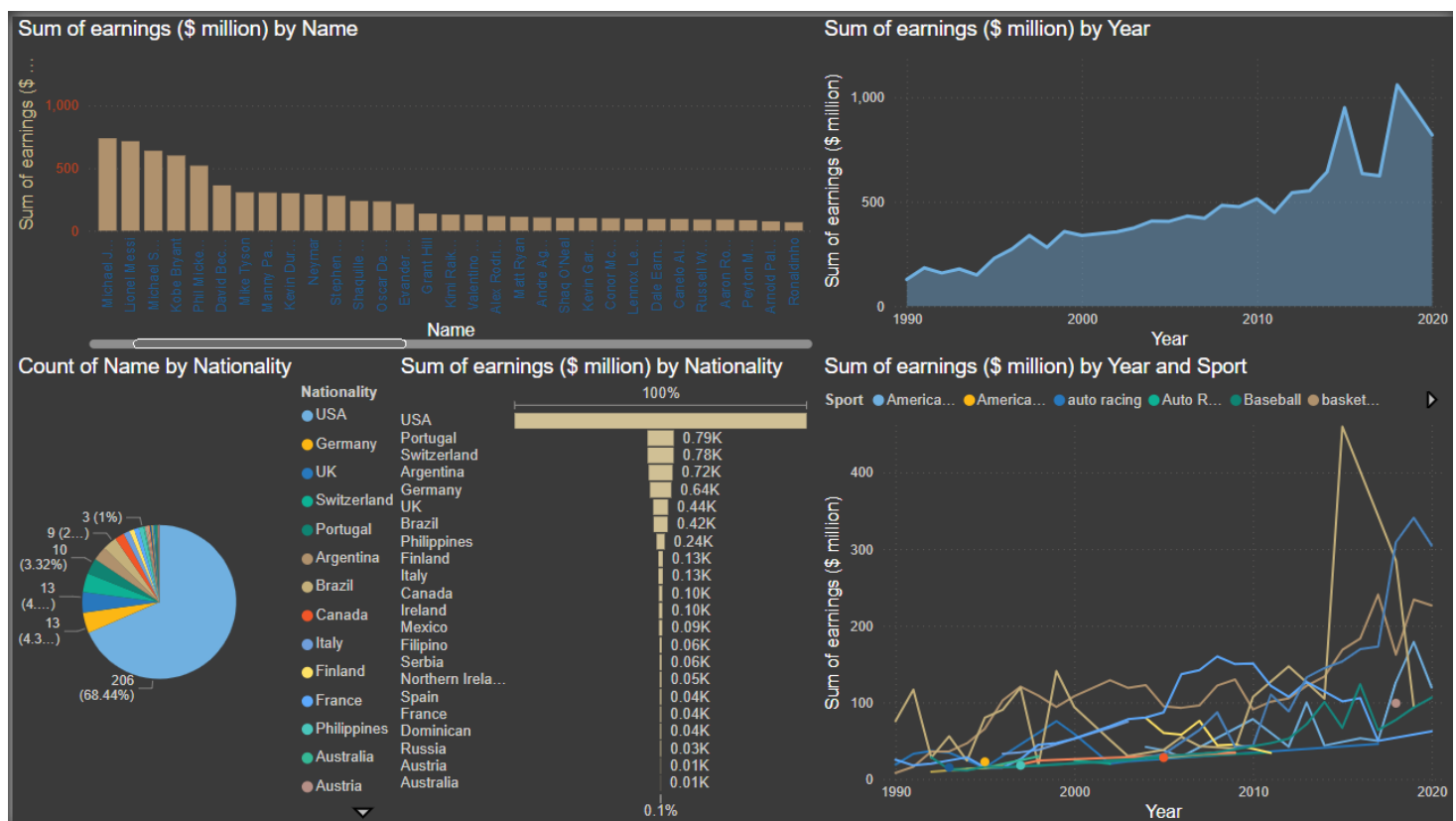
Dataset Source: <https://mavenanalytics.io/data-playground?page=2&pageSize=5>



4. Forbes Highest Paid Athletes (1990-2020).

Dataset source: <https://www.kaggle.com/datasets/parulpandey/forbes-highest-paid-athletes-19902019>

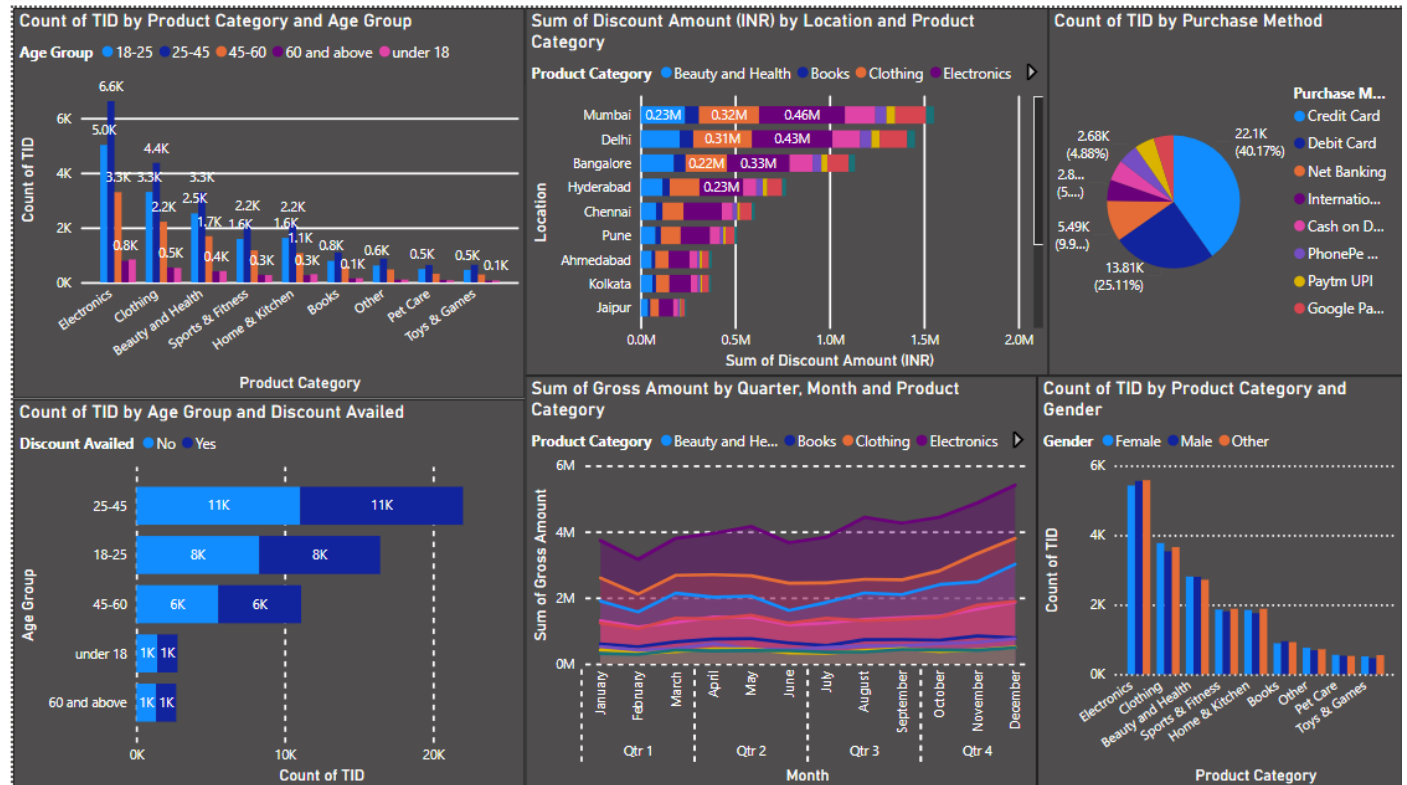
Tool used: MS Excel, MS Power BI



5. E-Commerce Data:

Dataset Source: <https://www.kaggle.com/datasets/shrishtimanja/ecommerce-dataset-for-data-analysis>

Tools Used: MS Power BI



6. Stock Market simulation (2022-2024)

Dataset source: <https://www.kaggle.com/datasets/samayashar/stock-market-simulation-dataset>

Tool used: Python (pandas, matplotlib, seaborn, plotly)

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
import matplotlib.ticker as ticker

# Load the dataset
df = pd.read_excel("synthetic_stock_data.xlsx")

# ----- 1. Trend Distribution (Improved) -----
plt.figure(figsize=(6, 4))
ax = sns.countplot(data=df, x='Trend', palette='Set2')

# Increase the number of ticks for accuracy
ax.yaxis.set_major_locator(ticker.MultipleLocator(50))
plt.title("Market Trend Distribution")
plt.xlabel("Trend")
plt.ylabel("Count")
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()

# ----- 2. Sector-wise Metrics -----
sector_summary = df.groupby('Sector').agg({
    'Close': 'mean',
    'PE_Ratio': 'mean',
    'Dividend_Yield': 'mean'
}).round(2).reset_index()

# Close Price
fig1 = px.bar(sector_summary, x='Sector', y='Close', title='Average Close Price by Sector',
              color='Sector', text='Close')
fig1.update_layout(yaxis=dict(tickmode="linear", dtick=5)) # Increase y-axis tick intervals
fig1.show()

# PE Ratio
fig2 = px.bar(sector_summary, x='Sector', y='PE_Ratio', title='Average PE Ratio by Sector',
              color='Sector', text='PE_Ratio')
fig2.update_layout(yaxis=dict(tickmode="linear", dtick=2)) # Increase y-axis tick intervals
fig2.show()

# Dividend Yield
fig3 = px.bar(sector_summary, x='Sector', y='Dividend_Yield', title='Average Dividend Yield by Sector',
              color='Sector', text='Dividend_Yield')
fig3.update_layout(yaxis=dict(tickmode="linear", dtick=0.5)) # Increase y-axis tick intervals
fig3.show()
```



```

# ----- 3. Sentiment Score by Sector -----
sentiment_summary = df.groupby('Sector')['Sentiment_Score'].mean().reset_index().sort_values(by='Sentiment_Score', ascending=False)

plt.figure(figsize=(8, 4))
ax = sns.barplot(data=sentiment_summary, x='Sentiment_Score', y='Sector', palette='coolwarm')

# Increase the number of ticks for better accuracy
ax.xaxis.set_major_locator(ticker.MultipleLocator(0.1))
plt.title("Average Sentiment Score by Sector")
plt.xlabel("Avg Sentiment Score")
plt.ylabel("Sector")
plt.grid(axis='x', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()

# ----- 4. Sentiment vs Close Price -----
plt.figure(figsize=(6, 4))
ax = sns.scatterplot(data=df, x='Sentiment_Score', y='Close', hue='Sector', alpha=0.6)

# More ticks for better accuracy
ax.xaxis.set_major_locator(ticker.MultipleLocator(0.2))
ax.yaxis.set_major_locator(ticker.MultipleLocator(5))
plt.title("Sentiment Score vs Close Price")
plt.xlabel("Sentiment Score")
plt.ylabel("Close Price")
plt.grid(True, linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()

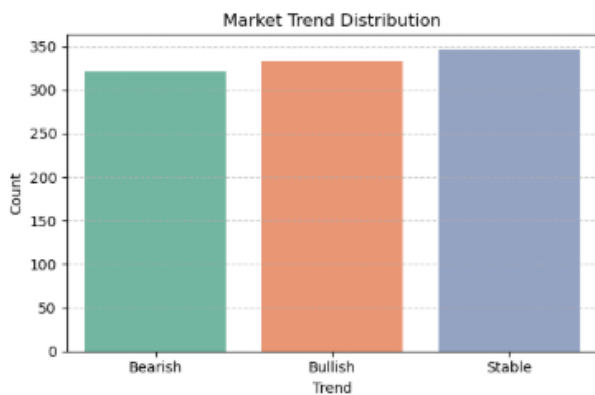
# ----- 5. Time Series Close Price (Sample Company) -----
company_name = df['Company'].unique()[0]
company_df = df[df['Company'] == company_name]

plt.figure(figsize=(10, 5))
ax = plt.plot(company_df['Date'], company_df['Close'], marker='o')

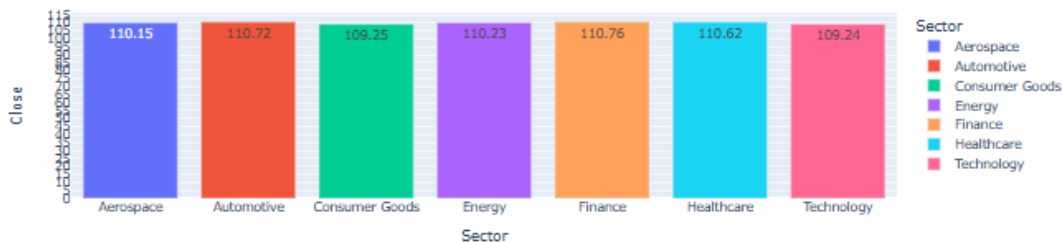
# Increase x-axis intervals for better readability
plt.xticks(rotation=45)
plt.locator_params(axis='x', nbins=10) # Increase the number of x-axis ticks

plt.title(f"Time Series Close Price: {company_name}")
plt.xlabel("Date")
plt.ylabel("Close Price")
plt.grid(True, linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()

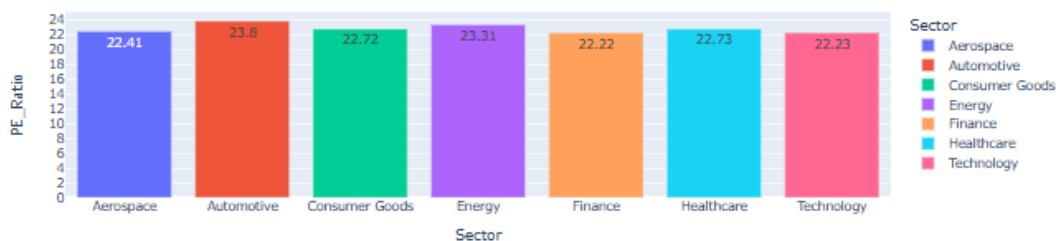
```

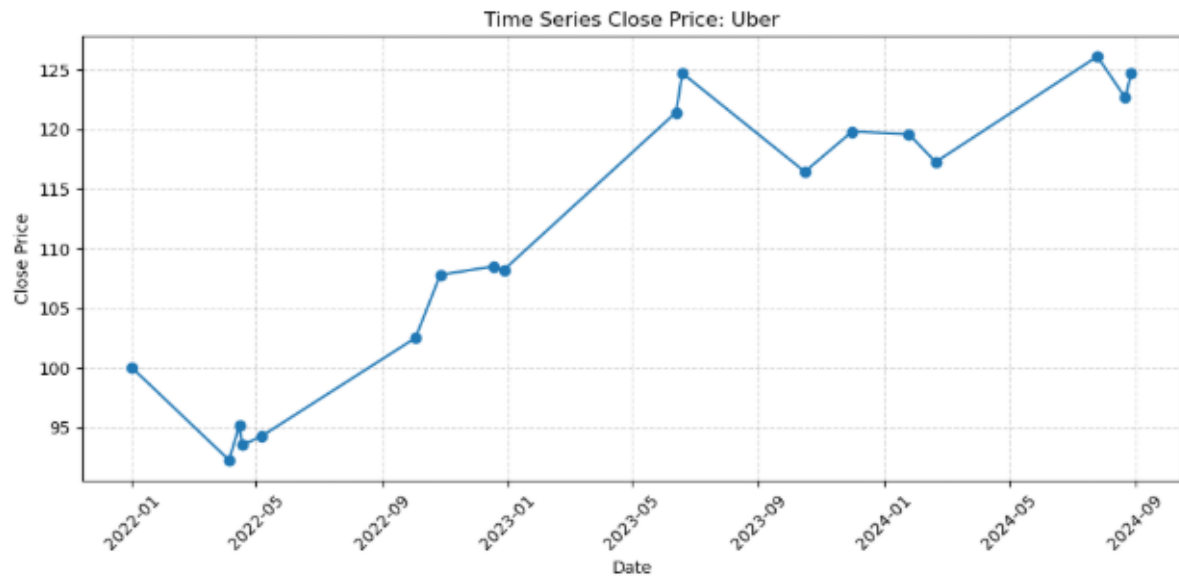


Average Close Price by Sector

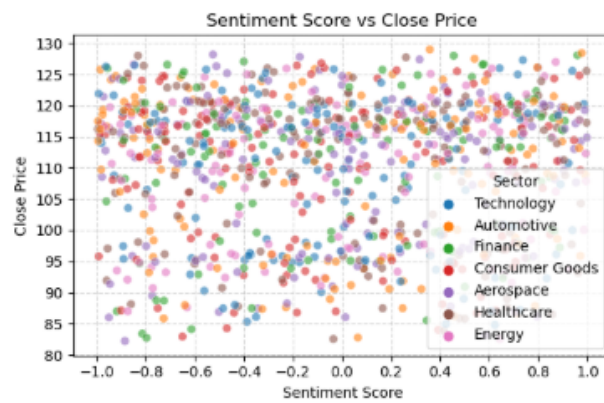
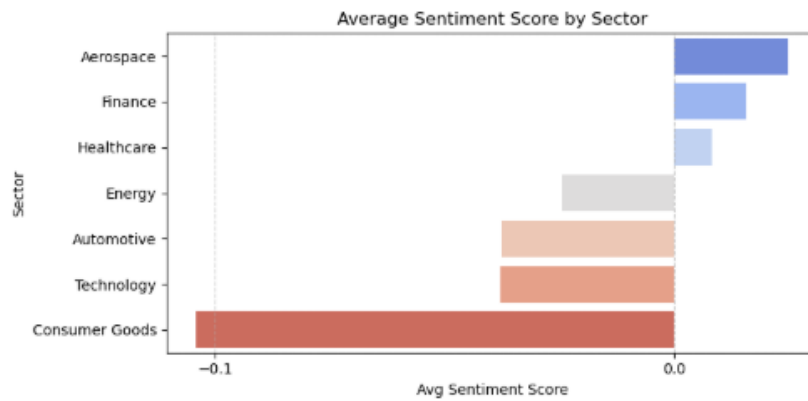


Average PE Ratio by Sector

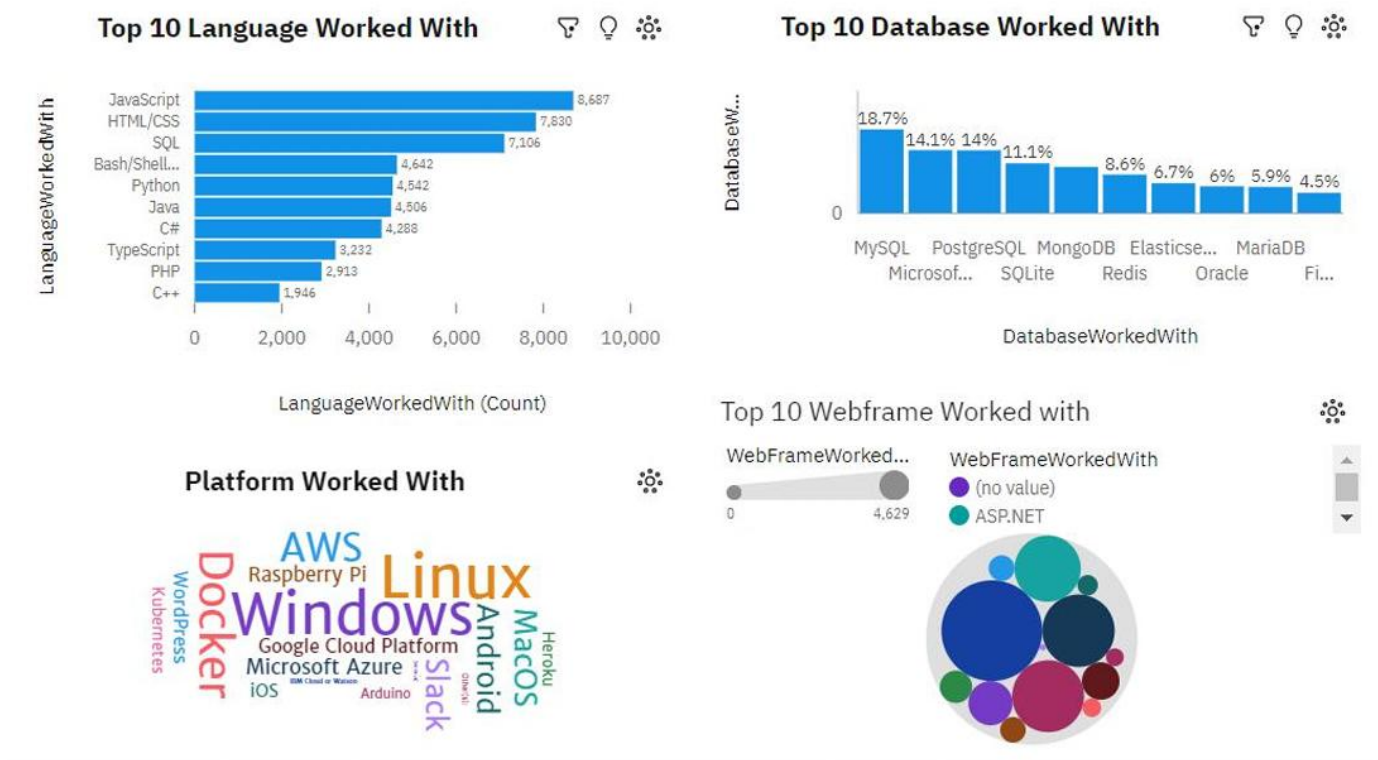
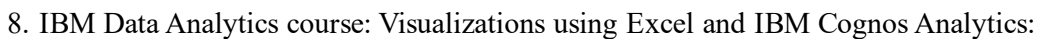


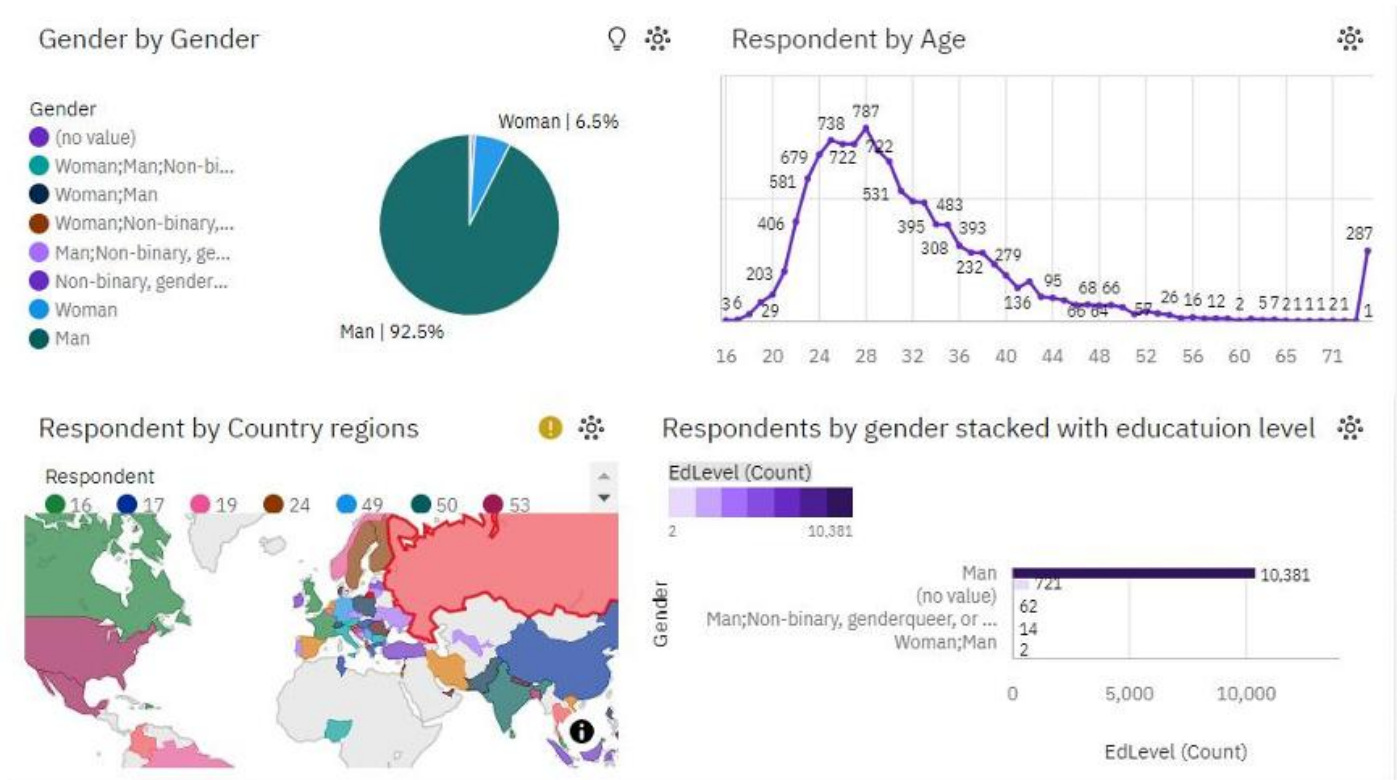


Average Dividend Yield by Sector



Tool used: MS Excel, MS Power BI



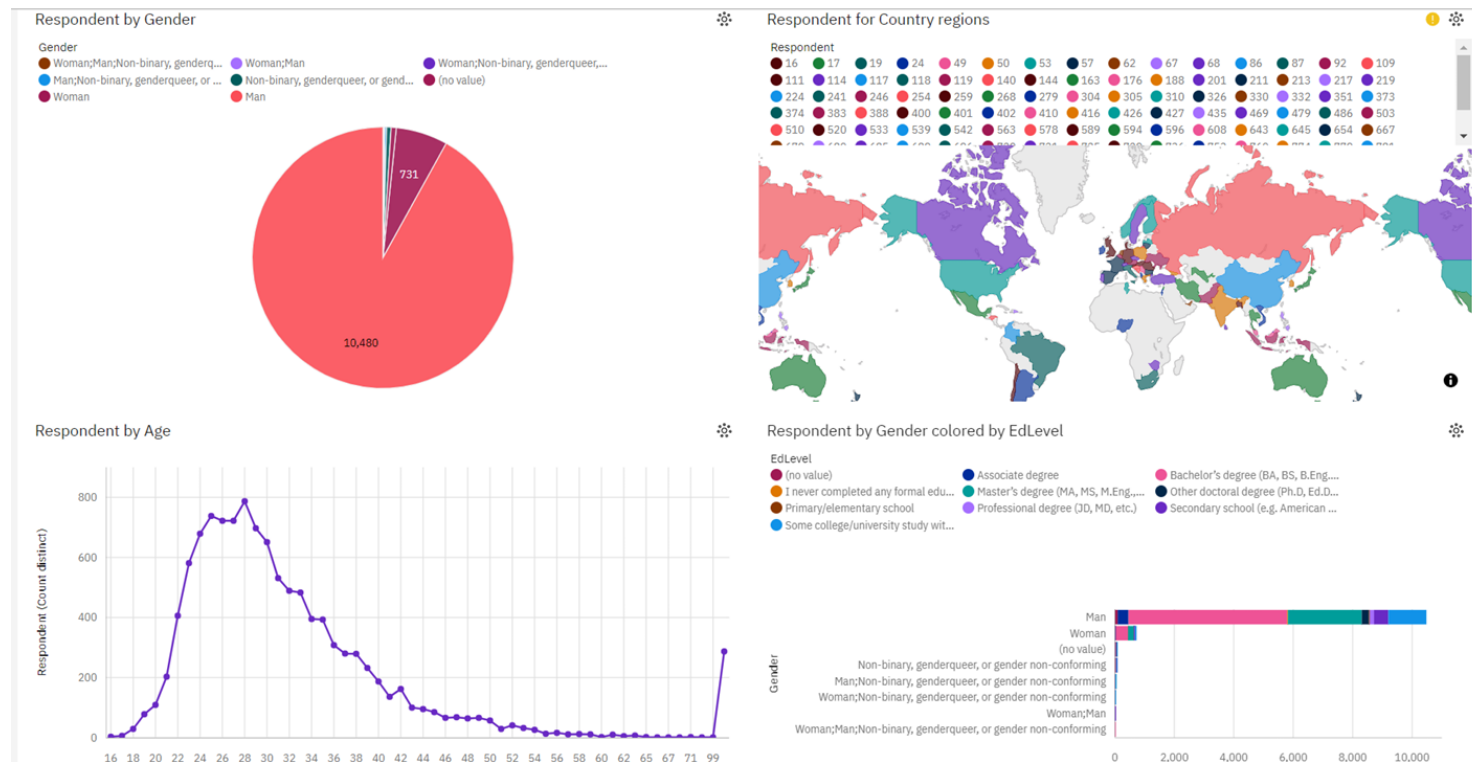


CERTIFICATION EARNED:

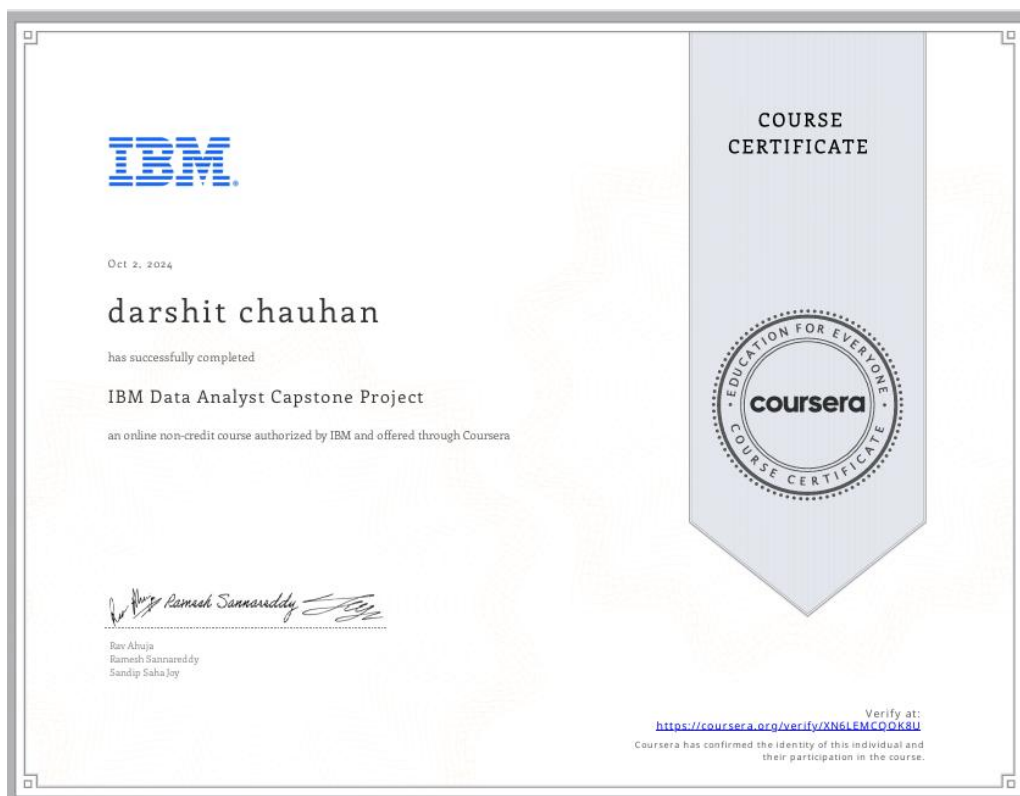


9. IBM Data Analytics course: CAPSTONE PROJECT.

DEMOGRAPHICS DASHBOARD:



CERTIFICATION EARNED:



10. NYC Air Bnb Data

Dataset Source: <https://www.kaggle.com/datasets/arianazmoudeh/airbnbopendata>

Tools Used: Python Data Science libraries

Source code:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
import dash_bootstrap_components as dbc
import warnings
warnings.filterwarnings("ignore", category=FutureWarning)

# Load the dataset
df = pd.read_csv('Airbnb_Open_Data.csv', dtype={25: str})
# Clean and convert price, service fee, and review rate number to numeric
for col in ['price', 'service fee', 'review rate number']:
    if col in df.columns:
        df[col] = (
            df[col]
            .astype(str)
            .str.replace(r'[^\\d.]', '', regex=True) # Remove $ and commas or other
characters
            .replace('', '0')
        )
        df[col] = pd.to_numeric(df[col], errors='coerce')

# CLEANING THE DATA SET
# FILLING BLANK SPACES WITH RESPECTIVE COLUMN APPROPRIATE VALUES
df['NAME'] = df['NAME'].fillna('NOT PROVIDED')
df['NAME'] = df['NAME'].replace('', 'NOT PROVIDED')
df['host_identity_verified'] = df['host_identity_verified'].fillna('unconfirmed')
df['host_identity_verified'] = df['host_identity_verified'].replace('', 'unconfirmed')
df['host name'] = df['host name'].fillna('UNDISCLOSED')
df['host name'] = df['host name'].replace('', 'UNDISCLOSED')
df['neighbourhood group'].replace('', pd.NA, inplace=True)
df['neighbourhood'].replace('', pd.NA, inplace=True)
df['country'] = df['country'].fillna('United States')
df['country'] = df['country'].replace('', 'United States')
df['country code'] = df['country code'].fillna('US')
df['country code'] = df['country code'].replace('', 'US')
df['instant_bookable'] = df['instant_bookable'].fillna('NOT SPECIFIED')
df['instant_bookable'] = df['instant_bookable'].replace('', 'NOT SPECIFIED')
df['cancellation_policy'] = df['cancellation_policy'].fillna('NO POLICY SPECIFIED')
df['cancellation_policy'] = df['cancellation_policy'].replace('', 'NO POLICY SPECIFIED')
df.replace('', pd.NA, inplace=True)
df['Construction year'] = df['Construction year'].fillna('NOT SPECIFIED')
df['service fee'] = df['service fee'].fillna('NOT APPLICABLE')
df['last review'] = df['last review'].fillna('NOT SPECIFIED')
df['review rate number'] = df['review rate number'].fillna(pd.NA)
df['calculated host listings count'] = df['calculated host listings count'].fillna(pd.NA)
df['house_rules'] = df['house_rules'].fillna('NO RULES SPECIFIED')
```

```

#Geographic Distribution of Listings
df['price_clean'] = df['price'].replace('[\$,]', '', regex=True).astype(float)

# Create map
fig = px.scatter_mapbox(df, lat="lat", lon="lon", color="room type",
                        size="price_clean", size_max=10, zoom=10,
                        mapbox_style="carto-positron",
                        title="Geographic Distribution of Listings")

fig.show()

# Distribution of Listings by Room Type
fig1 = px.pie(df, names='room type', title='Distribution of Listings by Room Type',
              color_discrete_sequence=px.colors.sequential.RdBu)

fig1.show()

#Top Neighbourhoods by Listing Count
top_neighbourhoods = df['neighbourhood'].value_counts().nlargest(10).reset_index()
top_neighbourhoods.columns = ['neighbourhood', 'count']

fig2 = px.bar(top_neighbourhoods, x='neighbourhood', y='count',
               title='Top 10 Neighbourhoods by Number of Listings',
               color='count', color_continuous_scale='Bluered')

fig2.show()

#Price Distribution
# Clean price column
df['price_clean'] = df['price'].replace('[\$,]', '', regex=True).astype(float)
fig3 = px.histogram(
    df,
    x='price_clean',
    nbins=100,
    title='Price Distribution of Listings',
    color_discrete_sequence=['indianred'])

# Add borders to the bars
fig3.update_traces(marker_line_color='black', marker_line_width=1)
fig3.update_layout(
    xaxis_title='Price ($)',
    yaxis_title='Number of Listings')
fig3.show()

#Price vs Review Rating
df['price_clean'] = df['price'].replace('[\$,]', '', regex=True).astype(float)
df['review rate number'] = pd.to_numeric(df['review rate number'], errors='coerce')

# Scatter Plot: Price vs Review Rating
fig = px.scatter(df, x='review rate number', y='price_clean',
                 title='Price vs Review Rating',
                 color='room type', opacity=0.6,
                 color_discrete_sequence=px.colors.qualitative.Set1)
fig.update_layout(xaxis_title='Review Rating', yaxis_title='Price ($)')
fig.show()

#Average Price by Room Type
# Compute average price by room type
avg_price = df.groupby('room type')['price_clean'].mean().reset_index()
# Plot

```

```

fig = px.bar(avg_price, x='room type', y='price_clean',
             title='Average Price by Room Type',
             color='price_clean', color_continuous_scale='Agsunset')
fig.update_layout(yaxis_title='Average Price ($)')
fig.show()

#Monthly Trend of Review
df['last review'] = pd.to_datetime(df['last review'], errors='coerce')
# Resample reviews per month
monthly_reviews = df.set_index('last review').resample('M')['number of reviews'].sum()
# Plot
fig = px.line(monthly_reviews, title='Monthly Trend of Reviews Over Time')
fig.update_layout(xaxis_title='Date', yaxis_title='Number of Reviews')
fig.show()

# Host Listing Strategy
df['price_clean'] = df['price'].replace('[\$,]', '', regex=True).astype(float)

# Plot histogram of listings per host
fig = px.histogram(df, x='calculated host listings count',
                  title='Distribution of Listings per Host',
                  nbins=50, color_discrete_sequence=['mediumseagreen'])
fig.update_layout(xaxis_title='Number of Listings per Host', yaxis_title='Host Count')
fig.show()

```

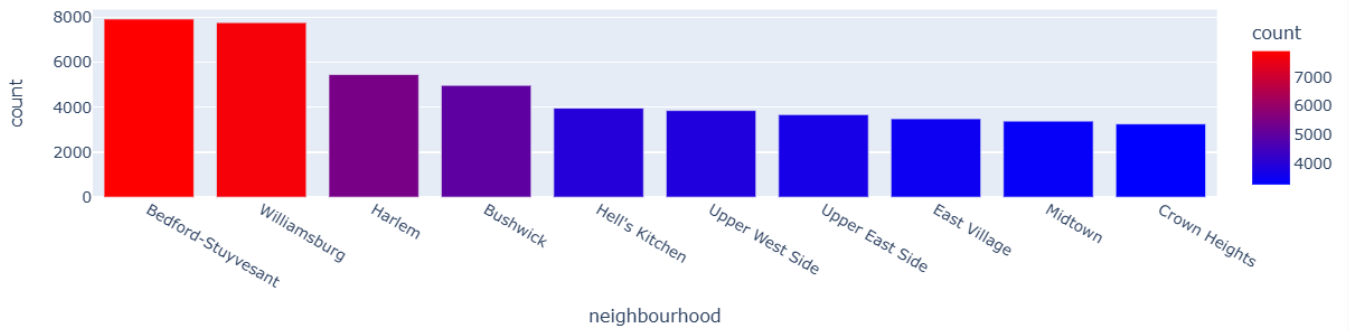
Geographic Distribution of Listings



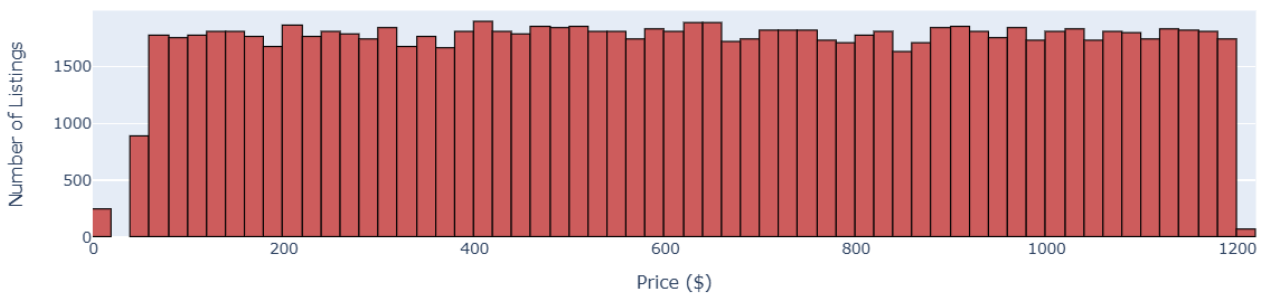
Distribution of Listings by Room Type



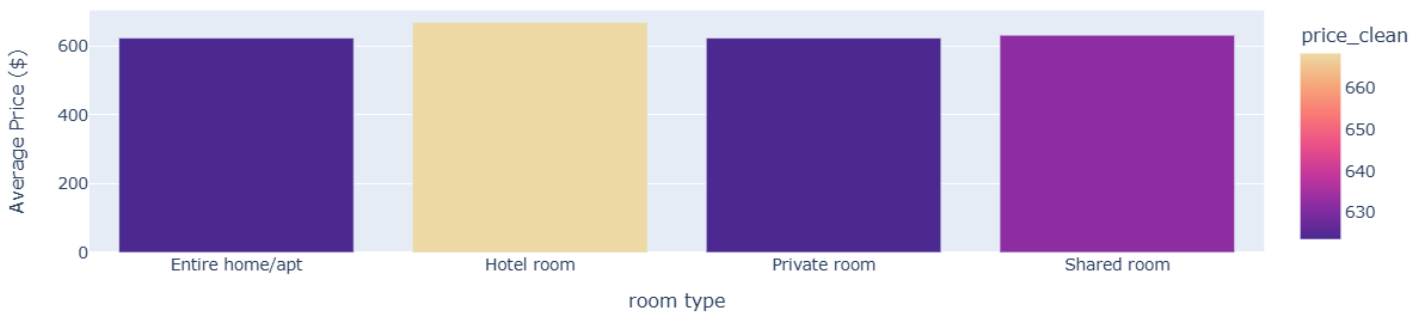
Top 10 Neighbourhoods by Number of Listings



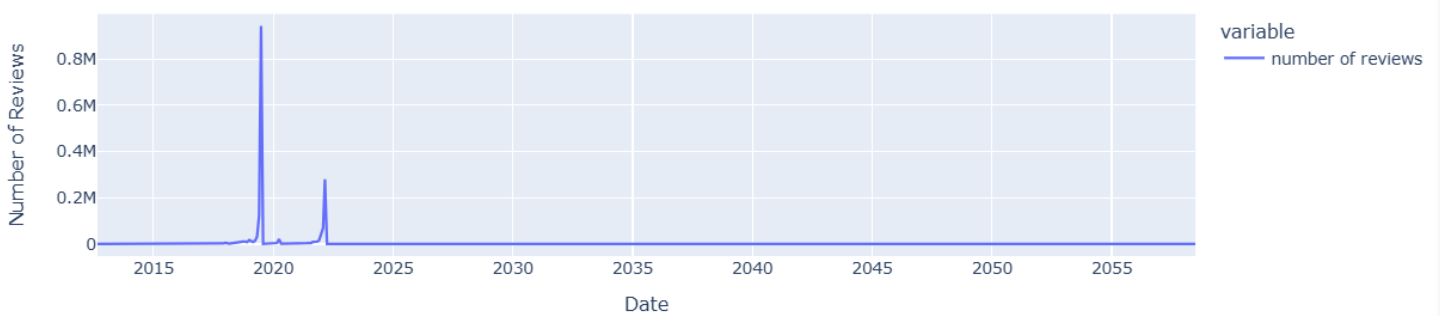
Price Distribution of Listings



Average Price by Room Type



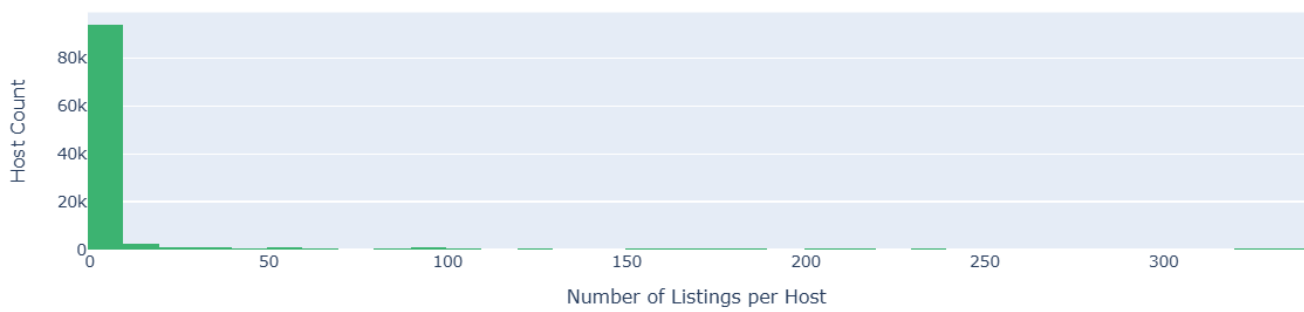
Monthly Trend of Reviews Over Time



Price vs Review Rating

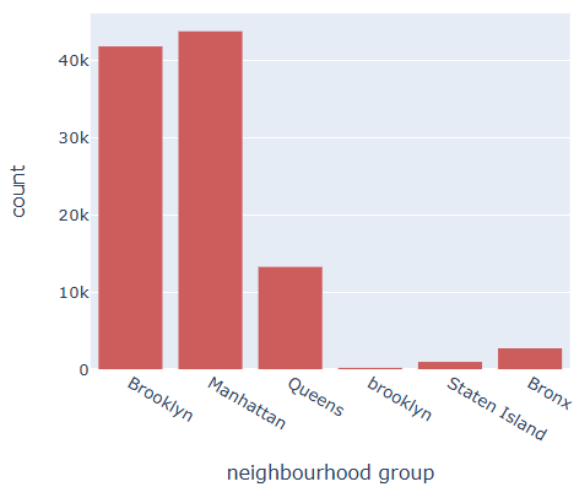


Distribution of Listings per Host

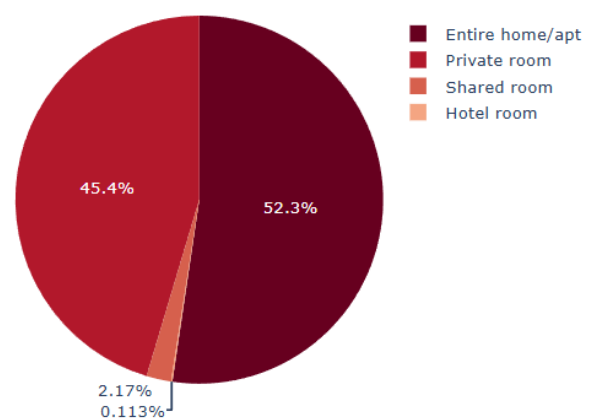


Airbnb Data Dashboard

Listings by Region



Room Type Distribution

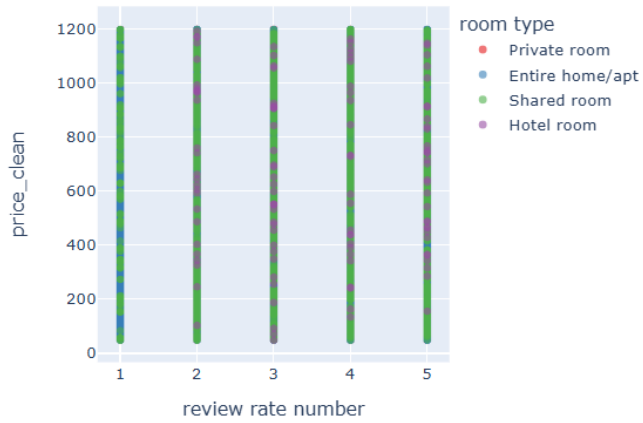


Price vs Review Rating

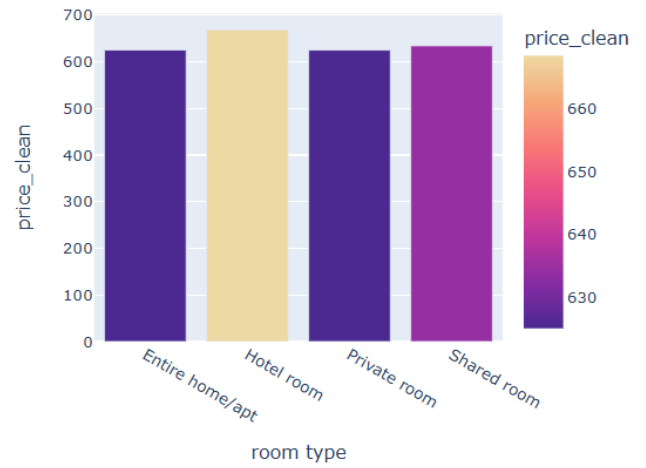
Average Price by Room Type



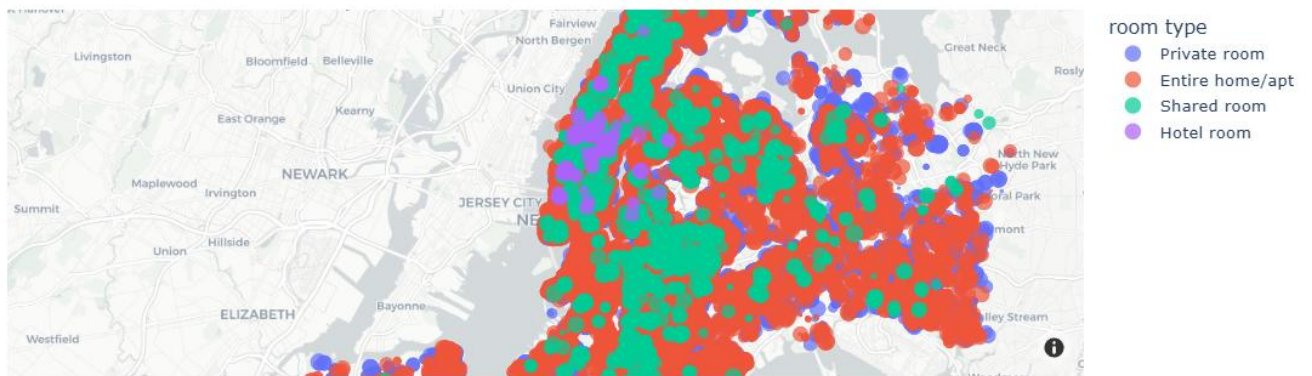
Price vs Review Rating



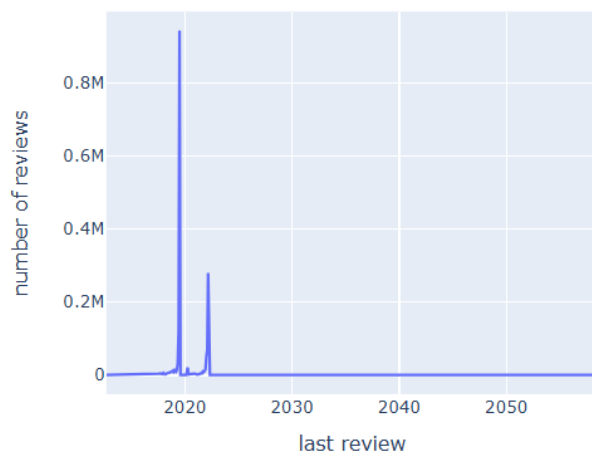
Average Price by Room Type



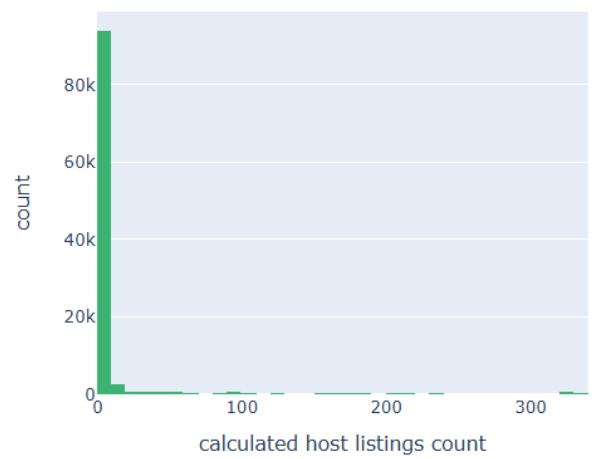
Geographic Distribution of Listings



Monthly Trend of Reviews Over Time



Distribution of Listings per Host



11. Crime Data(2020-Present)

Dataset Source: <https://www.kaggle.com/datasets/sonawanelalitsunil/crime-trends-2020present>

Tools Used: Python Data Science libraries.

Source code:

```
import pandas as pd
import plotly.express as px
import matplotlib.pyplot as plt
# Load the dataset
df = pd.read_csv('Crime_Data_from_2020_to_Present.csv')
# Fill missing values
df['Weapon Used Cd'] = df['Weapon Used Cd'].fillna('No Weapon used')
df['Weapon Desc'] = df['Weapon Desc'].fillna('No weapon used')
# Save cleaned data if needed
df.to_csv("cleaned_crime_data.csv", index=False)
df['Date Rptd'] = pd.to_datetime(df['Date Rptd'], format="%m/%d/%Y %I:%M:%S %p", errors='coerce')

# --- 1. Most Common Crimes by Area ---
crime_area = df['AREA NAME'].value_counts().nlargest(10).reset_index()
crime_area.columns = ['Area', 'Crime Count']
fig1 = px.bar(crime_area, x='Area', y='Crime Count', color='Crime Count',
              title='Top 10 Areas with Most Reported Crimes')
fig1.show()

# --- 2. Crime Trend Over Time ---
# Fix: Convert to string after to_period
df['Month'] = df['Date Rptd'].dt.to_period('M').astype(str)
monthly_trend = df.groupby('Month').size().reset_index(name='Crime Count')
fig2 = px.line(monthly_trend, x='Month', y='Crime Count', markers=True,
               title='Monthly Crime Trend',
               labels={'Month': 'Month', 'Crime Count': 'Number of Crimes'})
fig2.update_layout(yaxis_range=[0,12000], xaxis_tickangle=45)
fig2.show()

# --- 3. Time of Day When Crimes Occur ---
df['Hour'] = df['TIME OCC'].astype(str).str.zfill(4).str[:2].astype(int)
hourly_trend = df['Hour'].value_counts().sort_index().reset_index()
hourly_trend.columns = ['Hour', 'Crime Count']
fig3 = px.bar(hourly_trend, x='Hour', y='Crime Count', color='Crime Count',
              title='Crime Frequency by Hour of the Day')
fig3.show()

# --- 4. Most Common Premises for Crime ---
premises = df['Premis Desc'].value_counts().nlargest(10).reset_index()
premises.columns = ['Premises', 'Crime Count']
fig4 = px.bar(premises, x='Premises', y='Crime Count', color='Crime Count',
              title='Top 10 Premises Where Crimes Occur')
fig4.show()

# --- 5. Weapon Usage Analysis ---
weapon_usage = df['Weapon Desc'].value_counts().nlargest(10).reset_index()
weapon_usage.columns = ['Weapon', 'Count']
fig5 = px.pie(weapon_usage, names='Weapon', values='Count',
              title='Top 10 Weapons Used in Crimes')
```

```
fig5.show()
```

```
###6. premises vs weapons used by type---
```

```
# Filter out rows where weapons were used (excluding 'No weapon used')
```

```
weapon_df = df[df['Weapon Desc'] != 'No weapon used']
```

```
# Group by Premises and Weapon Description
```

```
weapon_premise = weapon_df.groupby(['Premis Desc', 'Weapon Desc']).size().reset_index(name='Count')
```

```
# Optional: Limit to Top 10 Premises and Top 10 Weapons for clarity
```

```
top_premises = weapon_premise.groupby('Premis Desc')['Count'].sum().nlargest(10).index
```

```
top_weapons = weapon_premise.groupby('Weapon Desc')['Count'].sum().nlargest(10).index
```

```
filtered = weapon_premise[weapon_premise['Premis Desc'].isin(top_premises) &  
                           weapon_premise['Weapon Desc'].isin(top_weapons)]
```

```
# Pivot for heatmap
```

```
pivot = filtered.pivot(index='Premis Desc', columns='Weapon Desc', values='Count').fillna(0)
```

```
# Heatmap using plotly
```

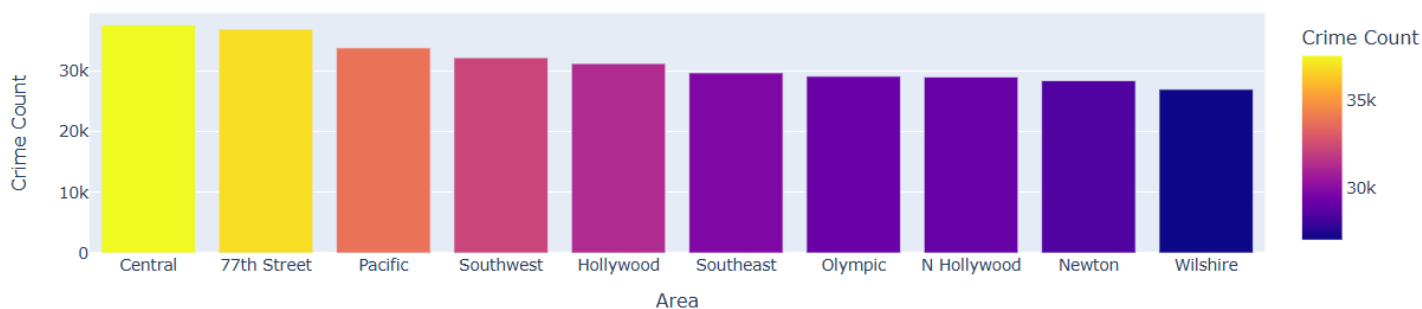
```
import plotly.graph_objects as go
```

```
fig = go.Figure(data=go.Heatmap(  
    z=pivot.values,  
    x=pivot.columns,  
    y=pivot.index,  
    colorscale='Reds',  
    hoverongaps=False  
))
```

```
fig.update_layout(  
    title='Heatmap: Premises vs Weapon Used',  
    xaxis_title='Weapon Description',  
    yaxis_title='Premises Description')  
fig.show()
```

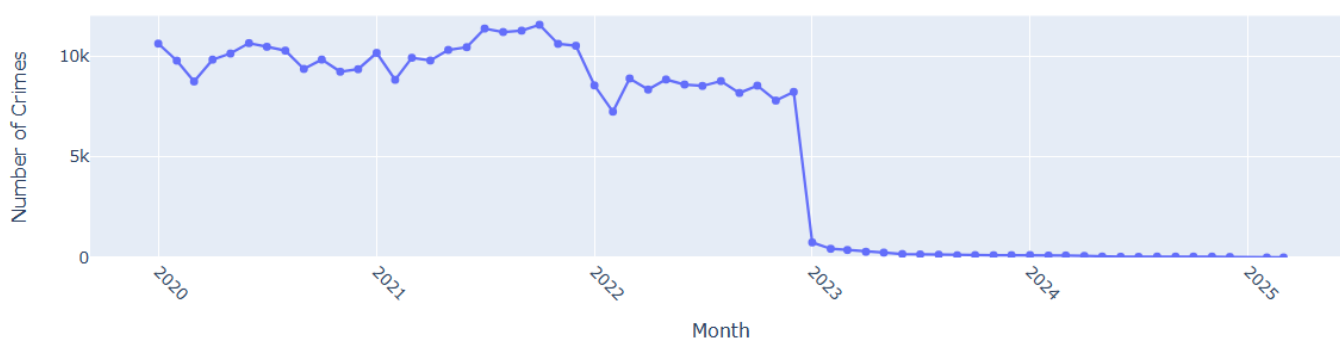
📷 🔍 + 📄 🗨 + - 🔍 🏠 📊

Top 10 Areas with Most Reported Crimes

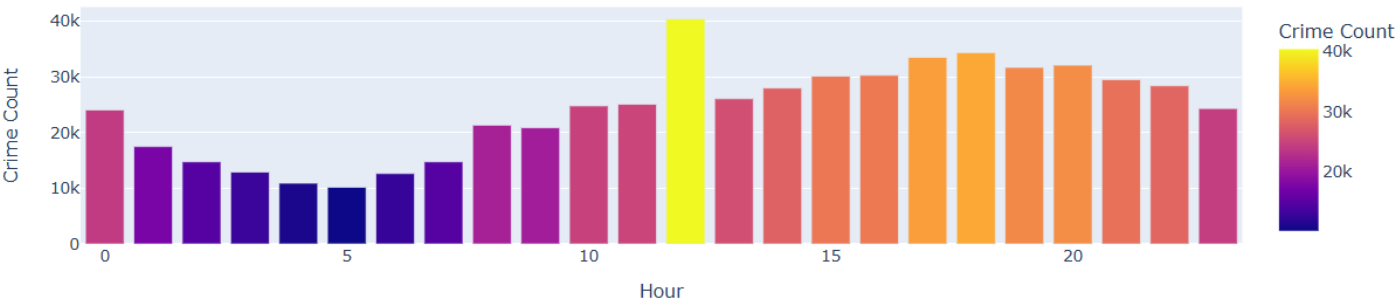


📷 🔍 + 📄 🗨 + - 🔍 🏠 📊

Monthly Crime Trend



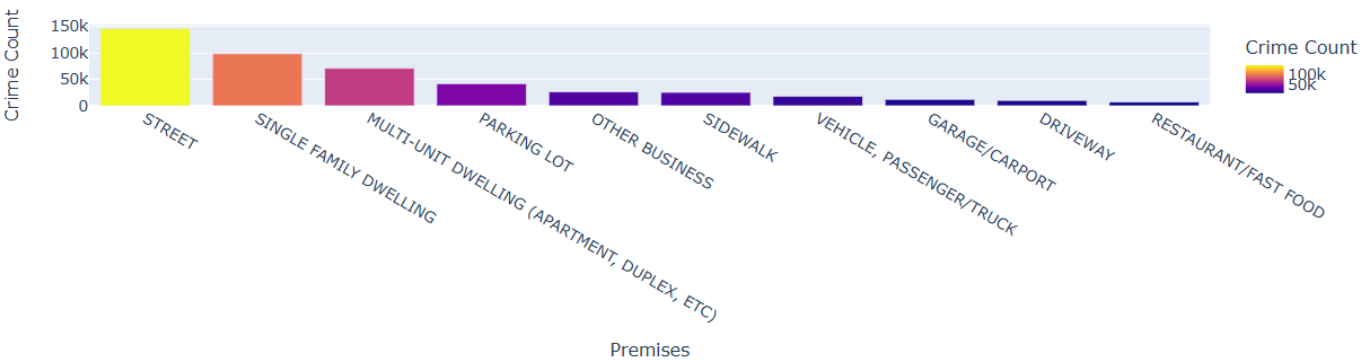
Crime Frequency by Hour of the Day



Top 10 Weapons Used in Crimes



Top 10 Premises Where Crimes Occur



Heatmap: Premises vs Weapon Used

